



PEMROGRAMAN WEB



www.google.com



PENGGUNA

1. SELECT DATA

- Menampilkan data tabel pengguna, pertama-tama menyiapkan back end terlebih dahulu yaitu berbasis php untuk pengambilan data
- Install axios. Jika sudah ada di package JSON, maka tidak perlu dilakukan install.
- Cara menarik data ada dua jenis yaitu POST dan GET
- POST menyimpan parameter yang di kirim dalam bentuk body, sedangkan GET menyimpan parameter pada url
- Hasil yang dikembalikan yaitu sebuah data berbentuk JSON
- Cara select data pengguna yaitu membuat function dimana pada contoh menggunakan metode GET. Hasil yang diperoleh disimpan pada variabel setter dan getter.

```
const Pengguna = () => {  
  const [user, setUser] = useState([]);  
  const fetchDataPengguna = async () => {  
    const url = "http://localhost/codebackendweb/selectpengguna.php";  
    const response = await axios.get(url);  
    setUser(response.data.DATA);  
  };  
};
```

- Lakukan pemanggilan fungsi untuk pertama kali component di load

```
useEffect(() => {  
  fetchDataPengguna();  
}, []);
```

- Pada component pengguna, buat tabel dan lakukan perulangan pada table body

```

<Table.Root size="sm" interactive>
  <Table.Header>
    <Table.Row>
      <Table.ColumnHeader>Nama</Table.ColumnHeader>
      <Table.ColumnHeader>Username</Table.ColumnHeader>
      <Table.ColumnHeader>Password</Table.ColumnHeader>
    </Table.Row>
  </Table.Header>
  <Table.Body>
    {user.map((item) => (
      <Table.Row key={item.id}>
        <Table.Cell>{item.nama}</Table.Cell>
        <Table.Cell>{item.username}</Table.Cell>
        <Table.Cell>{item.password}</Table.Cell>
      </Table.Row>
    ))}
  </Table.Body>
</Table.Root>

```

2. CREATE DATA

- Buat sebuah tombol tambah data dan ketika tombol tambah data di klik maka akan pindah ke halaman form tambah data.
- Langkah pertama buat route yang mengarah pada tambah data terlebih dahulu. Route masih di dalam nest route pada dashboard

```

const App = () => {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route index element={<Info />} />
        <Route path="pengguna" element={<Pengguna />} />
        <Route path="pengguna/tambah" element={<PenggunaCreate />} />
        <Route path="profil" element={<Profil />} />
      </Routes>
    </BrowserRouter>
  );
};

```

- Buat component baru di dalam folder pengguna. Berikan nama `penggunacreate.jsx`

```

return (
  <>
    <Box
      display="flex"
      flexDirection="column"
      width="100dvw"
      height="100dvh"
      justifyContent="center"
      alignItems="center"
    >
      <CardRoot width="50dvw" shadowColor="bg.emphasized" shadow="lg">
        <CardHeader>
          <CardTitle>
            <Text>Form Tambah Pengguna</Text>
          </CardTitle>
        </CardHeader>
        <CardBody>

```

- Pada pengguna, buat tombol sebagai link untuk pindah halaman ke form tambah

```

return (
  <>
    <Box padding="20px">
      <Heading size="xl" textAlign="center" padding="10px">
        Tabel Pengguna
      </Heading>
      <Box
        display="flex"
        flexDirection="row"
        justifyContent="right"
        padding="20px"
      >
        <Button variant="solid" bg="green.400" as={Link} to="tambah">
          <Text>Tambah Pengguna</Text>
        </Button>

```

- Jika sudah berhasil pindah halaman. Tahap berikutnya yaitu input dan kirim data pada form tambah. Sistem mirip dengan form login

Form Tambah Pengguna

Username

Password

Nama

Simpan Pengguna

Kembali

- Pada `penggunacreate`, buat setter getter untuk setiap input dan `usenavigate` untuk pindah halaman

```
const PenggunaCreate = () => {  
  const [username, setUsername] = useState("");  
  const [password, setPassword] = useState("");  
  const [nama, setNama] = useState("");  
  const navigate = useNavigate();
```

- Buat fungsi `handleSimpan` yang diarahkan ke php yang dituju.

```
const handleSimpan = async () => {  
  const url = "http://localhost/codebackendweb/insertpengguna.php";  
  const body = { username: username, password: password, nama: nama };  
  
  try {  
    const response = await axios.post(url, body);  
    if (response.data.STATUS === "BERHASIL") {  
      navigate("/dashboard/pengguna");  
    } else {  
      //ShowToast("INFO", "Simpan Gagal");  
      navigate("/dashboard/pengguna/tambah");  
    }  
  } catch (error) {  
    console.log(error);  
  }  
};
```

- Buat onClick ketika tombol simpan diklik
- Pada tombol kembali, lakukan pindah halaman ke menu dashboard/pengguna

```
<Button
  variant="solid"
  bg="green.400"
  onClick={() => handleSimpan()}
>
  Simpan Pengguna
</Button>
<Button variant="outline" as={Link} to="/dashboard/pengguna">
  Kembali
</Button>
```

- Tidak lupa untuk menambahkan onChange untuk setter setiap variabel

```
<Input
  type="text"
  border="1px solid grey"
  borderRadius="10px"
  placeholder="Username"
  onChange={(e) => {
    setUsername(e.target.value);
  }}
/>
<Input
  type="text"
  border="1px solid grey"
  borderRadius="10px"
  placeholder="Password"
  onChange={(e) => {
    setPassword(e.target.value);
  }}
/>
<Input
  type="text"
  border="1px solid grey"
  borderRadius="10px"
  placeholder="Nama"
  onChange={(e) => {
    setName(e.target.value);
  }}
/>
```

3. UPDATE DATA

- Kembali pada halaman pengguna. Buat setiap column memiliki aksi untuk update

Nama	Username	Password	Aksi	
AI ADMIN	admin123	ADMIN 1	Ubah	Hapus
AI 2 ADMIN	admin456	ADMIN 2	Ubah	Hapus
Admin Coba	admincoba	coba	Ubah	Hapus
admin test	admintest	test	Ubah	Hapus

Form Edit Pengguna

Edit Pengguna

Kembali

- Yang berbeda antara form update dan tambah yaitu ketika update terjadi oper parameter antar halaman. Ini bisa dilakukan oper parameter lewat route
- Langkah pertama yaitu buat halaman form update. Berikan nama penggunaupdate.jsx. Form sama persis modelnya seperti form tambah data.

```

return (
  <>
    <Box
      display="flex"
      flexDirection="column"
      width="100dvw"
      height="100dvh"
      justifyContent="center"
      alignItems="center"
    >
      <Toaster />
      <CardRoot width="50dvw" shadowColor="bg.emphasized" shadow="lg">
        <CardHeader>
          <CardTitle>
            <Text>Form Edit Pengguna </Text>
          </CardTitle>
        </CardHeader>
        <CardBody>

```

- Buat route untuk halaman update dimana terdapat route bisa oper parameter

```

const App = () => {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route index element={<Info />} />
        <Route path="pengguna" element={<Pengguna />} />
        <Route path="pengguna/tambah" element={<PenggunaCreate />} />
        <Route path="pengguna/edit/:id" element={<PenggunaUpdate />} />
        <Route path="profil" element={<Profil />} />
      </Routes>
    </BrowserRouter>
  );
};

```

- Pada halaman pengguna, pada tombol update lakukan pindah halaman disertakan dengan parameter yang dioper


```

<Table.Cell>
  <Box display="flex" flexDirection="row" gap="10px">
    <Button
      variant="solid"
      bg="blue.400"
      as={Link}
      to={`edit/${item.id}`}
    >
      <Text>Ubah</Text>
    </Button>

```

- Cara menangkap parameter pada form update bisa menggunakan useParams. Silahkan console log untuk cek berhasil oper atau tidak

```

const PenggunaUpdate = () => {
  const { id } = useParams();

```

- Karena update berarti mengambil data pengguna tertentu. Maka lakukan select data pada php yang dituju dan simpan pada variabel setter dan getter

```
const PenggunaUpdate = () => {
  const { id } = useParams();
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [nama, setNama] = useState("");
  const navigate = useNavigate();

  const fetchDataPengguna = async () => {
    const url =
    `http://localhost/codebackendweb/selectonepengguna.php?id=${id}`;
    const response = await axios.get(url);
    setUsername(response.data.DATA[0].username);
    setPassword(response.data.DATA[0].password);
    setNama(response.data.DATA[0].nama);
  };
};
```

```
useEffect(() => {
  fetchDataPengguna();
}, []);
```

- Setelah sudah simpan pada variabel maka bisa tampilkan pada form

```

<Input
  type="text"
  border="1px solid grey"
  borderRadius="10px"
  placeholder="Username"
  onChange={(e) => {
    setUsername(e.target.value);
  }}
  value={username}
/>
<Input
  type="text"
  border="1px solid grey"
  borderRadius="10px"
  placeholder="Password"
  onChange={(e) => {
    setPassword(e.target.value);
  }}
  value={password}
/>
<Input
  type="text"
  border="1px solid grey"
  borderRadius="10px"
  placeholder="Nama"
  onChange={(e) => {
    setNama(e.target.value);
  }}
  value={nama}
/>

```

- Langkah selanjutnya membuat tombol edit berfungsi untuk update data. Buat fungsi terlebih dahulu

```

const handleEdit = async () => {
  const url = "http://localhost/codebackendweb/updatepengguna.php";
  const body = { username: username, password: password, nama: nama, id: id };

  try {
    const response = await axios.post(url, body);
    if (response.data.STATUS === "BERHASIL") {
      navigate("/dashboard/pengguna");
    } else {
      ShowToast("INFO", "Simpan Gagal");
    }
  } catch (error) {
    console.log(error);
  }
};

```

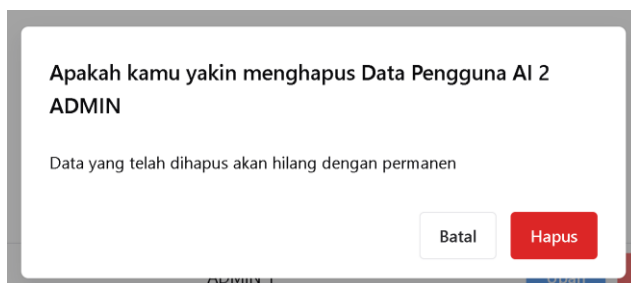
- Pada tombol edit, tambahkan onClick
- Pada tombol kembali berfungsi untuk kembali ke halaman dashboard/pengguna

```
<Button
  variant="solid"
  bg="blue.400"
  onClick={() => {
    handleEdit();
  }}
>
  Edit Pengguna
</Button>
<Button variant="outline" as={Link} to="/dashboard/pengguna">
  Kembali
</Button>
```

4. DELETE DATA

- Pada delete data, posisi ada di dekat tombol update, ketika diklik maka akan muncul dialog alert.

Nama	Username	Password	Aksi	
AI ADMIN	admin123	ADMIN 1	<button>Ubah</button>	<button>Hapus</button>
AI 2 ADMIN	admin456	ADMIN 2	<button>Ubah</button>	<button>Hapus</button>
Admin Coba	admincoba	coba	<button>Ubah</button>	<button>Hapus</button>
admin test	admintest	test	<button>Ubah</button>	<button>Hapus</button>



- Langkah pertama adalah tambahkan code alert dialog sebagai tombol delete

```

<Dialog.Root role="alertdialog">
  <Dialog.Trigger asChild>
    <Button variant="solid" bg="red.400" size="sm">
      Hapus
    </Button>
  </Dialog.Trigger>
  <Portal>
    <Dialog.Backdrop />
    <Dialog.Positioner>
      <Dialog.Content>
        <Dialog.Header>
          <Dialog.Title>
            Apakah kamu yakin menghapus Data Pengguna{" "}
            {item.nama}
          </Dialog.Title>
        </Dialog.Header>
        <Dialog.Body>
          <p>
            Data yang telah dihapus akan hilang dengan
            permanen
          </p>
        </Dialog.Body>
        <Dialog.Footer>
          <Dialog.ActionTrigger asChild>
            <Button variant="outline">Batal</Button>
          </Dialog.ActionTrigger>
          <Button
            colorPalette="red"
            onClick={() => {
              handleHapus(item.id);
            }}
          >
            Hapus
          </Button>
        </Dialog.Footer>
      </Dialog.Content>
    </Dialog.Positioner>
  </Portal>
</Dialog.Root>

```

- Ubah tombol pada alert dialog yaitu tombol hapus dan tombol batal
- Buat fungsi untuk handlehapus pada php yang dituju

```
const handleHapus = async (id) => {
  const url = `http://localhost/codebackendweb/deletepengguna.php?id=${id}`;

  try {
    const response = await axios.get(url);
    if (response.data.STATUS === "BERHASIL") {
      navigate("/dashboard/pengguna");
    } else {
      ShowToast("INFO", "Hapus Gagal");
    }
  } catch (error) {
    console.log(error);
  }

  await fetchDataPengguna();
};
```

- Tambahkan onClick pada tombol hapus

```
<Dialog.Footer>
  <Dialog.ActionTrigger asChild>
    <Button variant="outline">Batal</Button>
  </Dialog.ActionTrigger>
  <Button
    colorPalette="red"
    onClick={() => {
      handleHapus(item.id);
    }}
  >
    Hapus
  </Button>
</Dialog.Footer>
```

**SELAMAT ANDA SUDAH BERHASIL
MEMBUAT WEB BERBASIS REACT JS, CHAKRA UI V3 dan PHP**